

Implementation of Class Diagram

Experiment No: 03

Date: 19/09/2026

Title: Implementation of Class Diagram – Library Management System

■ Objective / Aim

To design and implement a Library Management System in Java that translates a UML class diagram into working code, demonstrating key object-oriented relationships such as inheritance, association, aggregation, and composition.

■ System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

Software

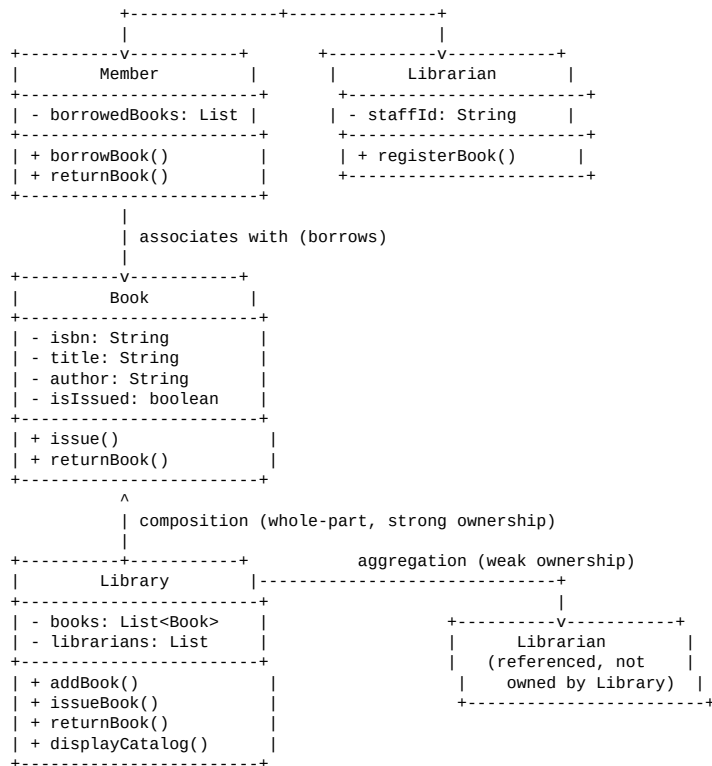
- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

■ Algorithm / Logic Description

1. Create a class **Book** with attributes: isbn, title, author, isIssued.
2. Create a class **Member** with attributes: memberId, name, and a list of borrowed Book objects (association).
3. Create a class **Librarian** with attributes: staffId, name.
4. Create a class **Library** that composes a list of Book objects (books cannot exist independently of the library) and aggregates a list of Librarian objects (librarians exist independently of the library).
5. Define methods in Library: addBook(), issueBook(), returnBook(), displayCatalog().
6. Define borrowBook() and returnBook() in Member to interact with the Library.
7. Create a base class **Person** with common attributes (name, id) and let Member and Librarian inherit from it.
8. Instantiate a Library, add Librarian(s) and Book(s), and create Member objects.
9. Perform issue/return transactions and display the catalog state before and after.

■ UML Class Diagram

```
+-----+
|         Person         |
+-----+
| # id: String           |
| # name: String         |
+-----+
| + displayDetails()     |
+-----+
```



Relationships shown:

- **Inheritance:** Member and Librarian inherit from Person.
- **Composition:** Library *owns* Book objects – books don't exist outside a library instance.
- **Aggregation:** Library *has* Librarians – librarians can exist and be reused independently.
- **Association:** Member borrows Book objects – a simple "uses" relationship.

■ Source Code

```

import java.util.ArrayList;
import java.util.List;

// Base class - Inheritance
abstract class Person {
    protected String id;
    protected String name;

    Person(String id, String name) {
        this.id = id;
        this.name = name;
    }

    public void displayDetails() {
        System.out.println("ID : " + id);
        System.out.println("Name : " + name);
    }
}

// Book class
class Book {
    private String isbn;
    private String title;
    private String author;
    private boolean isIssued;

    Book(String isbn, String title, String author) {
        this.isbn = isbn;
        this.title = title;
        this.author = author;
        this.isIssued = false;
    }

    public boolean issue() {

```

```

        if (!isIssued) {
            isIssued = true;
            return true;
        }
        return false;
    }

    public void returnBook() {
        isIssued = false;
    }

    public boolean isIssued() {
        return isIssued;
    }

    public String getTitle() {
        return title;
    }

    public String getIsbn() {
        return isbn;
    }

    public void displayInfo() {
        System.out.println(isbn + " | " + title + " by " + author
            + " | " + (isIssued ? "Issued" : "Available"));
    }
}

// Librarian class - Inheritance from Person (Aggregation with Library)
class Librarian extends Person {
    private String staffId;

    Librarian(String staffId, String name) {
        super(staffId, name);
        this.staffId = staffId;
    }

    public void registerBook(Library library, Book book) {
        library.addBook(book);
        System.out.println(name + " registered book: " + book.getTitle());
    }
}

// Member class - Inheritance from Person, Association with Book
class Member extends Person {
    private List<Book> borrowedBooks = new ArrayList<>();

    Member(String id, String name) {
        super(id, name);
    }

    public void borrowBook(Library library, String isbn) {
        Book book = library.issueBook(isbn);
        if (book != null) {
            borrowedBooks.add(book);
            System.out.println(name + " borrowed: " + book.getTitle());
        } else {
            System.out.println("Book unavailable for borrowing.");
        }
    }

    public void returnBook(Library library, String isbn) {
        for (Book b : borrowedBooks) {
            if (b.getIsbn().equals(isbn)) {
                library.returnBook(isbn);
                borrowedBooks.remove(b);
                System.out.println(name + " returned: " + b.getTitle());
                return;
            }
        }
        System.out.println(name + " does not have this book.");
    }
}

// Library class - Composition of Book, Aggregation of Librarian

```

```

class Library {
    private List<Book> books = new ArrayList<>(); // Composition
    private List<Librarian> librarians = new ArrayList<>(); // Aggregation

    public void addBook(Book book) {
        books.add(book);
    }

    public void addLibrarian(Librarian librarian) {
        librarians.add(librarian); // Library references, doesn't own lifecycle
    }

    public Book issueBook(String isbn) {
        for (Book b : books) {
            if (b.getIsbn().equals(isbn) && b.issue()) {
                return b;
            }
        }
        return null;
    }

    public void returnBook(String isbn) {
        for (Book b : books) {
            if (b.getIsbn().equals(isbn)) {
                b.returnBook();
                return;
            }
        }
    }

    public void displayCatalog() {
        System.out.println("----- Library Catalog -----");
        for (Book b : books) {
            b.displayInfo();
        }
    }
}

// Main class
public class LibrarySystemDemo {
    public static void main(String[] args) {

        Library library = new Library();

        // Librarian registers books (aggregation + composition in action)
        Librarian librarian = new Librarian("L01", "Mr. Sharma");
        library.addLibrarian(librarian);

        librarian.registerBook(library, new Book("B001", "Clean Code", "Robert C. Martin"));
        librarian.registerBook(library, new Book("B002", "Effective Java", "Joshua Bloch"));

        System.out.println();
        library.displayCatalog();

        // Member borrows and returns books (association in action)
        Member member = new Member("M01", "Ananya");

        System.out.println("\n----- Borrowing -----");
        member.borrowBook(library, "B001");
        member.borrowBook(library, "B001"); // already issued

        System.out.println("\n----- Catalog After Borrowing -----");
        library.displayCatalog();

        System.out.println("\n----- Returning -----");
        member.returnBook(library, "B001");

        System.out.println("\n----- Catalog After Returning -----");
        library.displayCatalog();
    }
}

```

■ Compilation & Execution Commands

Save the source code exactly as:

LibrarySystemDemo.java

Compile the program:

javac LibrarySystemDemo.java

Run the compiled program:

java LibrarySystemDemo

■ Output Screen

```
Mr. Sharma registered book: Clean Code
Mr. Sharma registered book: Effective Java

----- Library Catalog -----
B001 | Clean Code by Robert C. Martin | Available
B002 | Effective Java by Joshua Bloch | Available

----- Borrowing -----
Ananya borrowed: Clean Code
Book unavailable for borrowing.

----- Catalog After Borrowing -----
B001 | Clean Code by Robert C. Martin | Issued
B002 | Effective Java by Joshua Bloch | Available

----- Returning -----
Ananya returned: Clean Code

----- Catalog After Returning -----
B001 | Clean Code by Robert C. Martin | Available
B002 | Effective Java by Joshua Bloch | Available
```

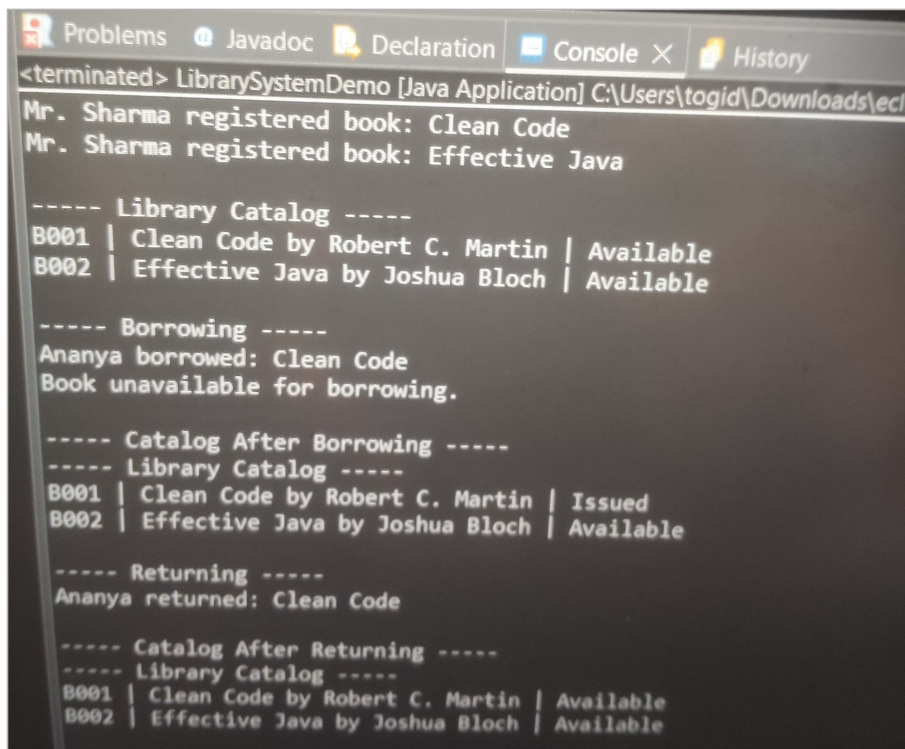


Fig: Console output of LibrarySystemDemo in Eclipse IDE

■ Result

The Library Management System was successfully implemented in Java by translating a UML class diagram into code. The program demonstrates inheritance (Member/Librarian from Person), composition (Library–Book), aggregation (Library–Librarian), and association (Member–Book) as distinct object-oriented relationships.